

Git & GitHub Terminal Guide 2026

Standard Version Control Protocol @ Illarek-Lab

johnkbarrera.github.io | john.barrera@unmsm.edu.pe

1 Introduccion

Esta guia provee el protocolo estandar para subir un proyecto local (Android, web o cualquier otro) a un repositorio remoto en GitHub utilizando exclusivamente la terminal. Es el flujo recomendado para entornos profesionales y para entregas academicas formales.

Axioma de Version Control

Todo proyecto profesional debe estar bajo control de versiones desde el primer commit. Git registra el historial completo del codigo, permite trabajo colaborativo y protege contra perdidas accidentales. GitHub es el servidor remoto que aloja ese historial y lo hace accesible al equipo.

2 Implementacion: Subir Proyecto a GitHub

2.1 Requisitos Previos

Hardware & Software Checklist

En la estacion: Git instalado (`git -version`)
Editor o IDE (Android Studio, VS Code)
Acceso a terminal (CMD, PowerShell, Bash)

En la nube: Cuenta activa en `github.com`
Personal Access Token (PAT) generado
Conexion estable a internet

2.2 Creacion del Repositorio en GitHub

Configuracion del Repositorio Remoto

URL:	<code>https://github.com</code> -> Boton "+" -> New repository
Repository name:	SanMarcosStore
Description:	App Android Material 3 (opcional)
Visibility:	Public / Private
☐ Add a README	(NO marcar)
☐ Add .gitignore	(NO marcar)
☐ Choose a license	(NO marcar)

Critico: el repositorio remoto debe quedar 100% vacio. Si se marcan las casillas adicionales, GitHub crea archivos que generaran conflictos al hacer el primer push desde el cliente local.

Tras crear el repo, copiar la URL que termina en `.git`:

URL del Repositorio Remoto

```
https://github.com/tu_usuario/SanMarcosStore.git
```

2.3 Configuración Inicial de Git

Ejecución única por estación de trabajo. Define la identidad asociada a cada commit.

Comandos: Identidad del Usuario

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu_correo@ejemplo.com"

# Verificar configuración
git config --global --list
```

Nota: el correo debe coincidir con el de la cuenta GitHub para que los commits queden asociados al perfil del usuario.

2.4 Apertura de la Terminal en el Proyecto

2.4.1 Opción A: Terminal de Android Studio

1. En Android Studio: **View** → **Tool Windows** → **Terminal**
2. Atajo: Alt + F12
3. La terminal abre automáticamente en la raíz del proyecto

2.4.2 Opción B: Terminal del Sistema

1. Abrir CMD / PowerShell / Bash
2. Navegar a la carpeta del proyecto: `cd ruta/al/proyecto`
3. Verificar ubicación con: `pwd`

2.5 Inicialización y Primer Push

Ejecutar los siguientes comandos en orden, uno por uno, validando el resultado de cada paso antes de continuar.

Pipeline de Inicializacion

```
# 1. Inicializar Git en el proyecto
git init

# 2. Renombrar la rama principal a 'main' (estandar moderno)
git branch -M main

# 3. Agregar todos los archivos al staging area
git add .

# 4. Crear el primer commit
git commit -m "Initial commit"

# 5. Conectar con el repositorio remoto
git remote add origin https://github.com/tu_usuario/SanMarcosStore.git

# 6. Subir el codigo a GitHub
git push -u origin main
```

Detalle del flag `-u`: establece la rama remota como upstream de la local. Solo se utiliza en el primer push. En commits posteriores basta con `git push`.

2.6 Autenticacion: Personal Access Token (PAT)

2.6.1 Generacion del Token

GitHub no acepta passwords tradicionales para operaciones Git desde 2021. Se requiere un PAT.

1. En GitHub → avatar (esquina superior derecha) → **Settings**
2. Panel lateral inferior → **Developer settings**
3. **Personal access tokens** → **Tokens (classic)**
4. Click **Generate new token (classic)**
5. Marcar el scope repo (permite push/pull en repositorios)
6. Click **Generate token** al final del formulario
7. **Copiar el token inmediatamente** (formato `ghp_aBcD1234...`)

2.6.2 Uso del Token en Terminal

Cuando la terminal solicite credenciales durante `git push`:

Verificacion de Credenciales

```
Username for 'https://github.com': tu_usuario_github
Password for 'https://tu_usuario@github.com': [PEGAR EL TOKEN AQUI]
```

Critico: no usar el password normal de la cuenta GitHub. El token reemplaza al password en operaciones desde linea de comandos.

2.7 Verificacion del Despliegue

Output Esperado de `git push`

```
Enumerating objects: 100, done.  
Counting objects: 100 % (100/100), done.  
Writing objects: 100 % (100/100), 1.2 MiB  
  
* [new branch] main ->main  
branch 'main' set up to track 'origin/main'.
```

Refrescar el repositorio en el navegador (F5) para confirmar que todos los archivos del proyecto estan visibles en GitHub.

2.8 Configuracion del Archivo .gitignore

2.8.1 Proposito

El archivo `.gitignore` indica a Git que archivos o carpetas excluir del tracking. Es critico para proyectos Android, ya que carpetas como `build/` y `.gradle/` contienen archivos generados que no deben subirse al repositorio.

2.8.2 Contenido Recomendado para Android

Contenido: `.gitignore` (Raiz del Proyecto)

```
*.iml  
.gradle  
/local.properties  
/.idea/  
.DS_Store  
/build  
/captures  
.externalNativeBuild  
.cxx  
local.properties  
app/build/
```

Nota: Android Studio genera este archivo automaticamente al crear un nuevo proyecto. Si no existe, debe crearse manualmente en la raiz antes del primer `git add`.

3 Flujo de Trabajo: Cambios Posteriores

Una vez configurado el repositorio, cualquier modificacion al codigo se sube en tres comandos.

Pipeline de Actualizacion

```
# 1. Revisar archivos modificados (opcional pero recomendado)
git status

# 2. Agregar todos los cambios al staging
git add .

# 3. Crear el commit con un mensaje descriptivo
git commit -m "Agregada pantalla de Carrito con FilterChips"

# 4. Subir cambios al repositorio remoto
git push
```

Convencion de mensajes: usar verbos en presente o pasado, en espanol o ingles, describiendo el cambio especifico. Evitar mensajes genericos como `update.` o `cambios`.

4 Diagrama de Flujo: Git Push Pipeline

Pipeline Completo de Despliegue

```
[ Working Directory ]  ->  [ git add ]  ->  [ Staging Area ]
      |                               |
[Codigo modificado ]           [ git commit ]
                               |
                               v
[ Repositorio Remoto ]  <-  [ git push ]  <-  [ Local Repository ]
      ^                               |
      |                               |
[ github.com ]  <-  [ HTTPS + PAT ]  <-  [ origin/main ]
```

5 Solucion de Problemas

Error	Solucion
Authentication failed	Se uso el password real en lugar del PAT. Generar token segun seccion 2.6 y usarlo como password
Updates were rejected	El repo remoto tiene archivos que el local no tiene. Ejecutar: <code>git pull origin main --allow-unrelated-histories</code> y luego <code>git push</code>
remote: Repository not found	URL del remote incorrecta. Verificar con <code>git remote -v</code> . Corregir con: <code>git remote set-url origin <URL_correcta></code>
fatal: not a git repository	Falto ejecutar <code>git init</code> en la raiz del proyecto. Volver al paso 2.5
Permission denied (publickey)	Se intento usar SSH sin clave configurada. Cambiar a HTTPS con: <code>git remote set-url origin https://github.com/...</code>
Suben carpetas build/ o .idea/	.gitignore mal configurado o creado despues del primer add. Ejecutar: <code>git rm -r -cached build/ .idea/</code> luego <code>commit + push</code>

refusing to merge unrelated histories	Se marco <code>.Add README.</code> al crear el repo. Ejecutar: <code>git pull origin main --allow-unrelated-histories</code> y resolver conflictos
nothing to commit, working tree clean	No hay cambios para commitear. Modificar al menos un archivo antes de <code>git add .</code>

6 Cheatsheet: Comandos Esenciales

Setup Unico (Por PC)

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu_correo@ejemplo.com"
```

Setup Unico (Por Proyecto)

```
git init
git branch -M main
git add .
git commit -m "Initial commit"
git remote add origin https://github.com/tu_usuario/SanMarcosStore.git
git push -u origin main
```

Cada Vez Que Haya Cambios

```
git add .
git commit -m "Mensaje del cambio"
git push
```

Comandos Utiles Adicionales

```
# Ver estado actual del repositorio
git status

# Ver historial de commits
git log --oneline

# Ver URL del repositorio remoto
git remote -v

# Descartar cambios locales no commiteados
git checkout -- .

# Clonar un repositorio existente
git clone https://github.com/usuario/proyecto.git
```

7 Consideraciones de Seguridad

Buenas Practicas y Limitaciones

- ✓ **Permite:** Versionar codigo fuente, archivos de configuracion, documentacion
- ✓ **Permite:** Trabajo colaborativo con multiples ramas y pull requests
- ✗ **No subir:** Archivos con credenciales, API keys, tokens, passwords
- ✗ **No subir:** Carpetas build/, node_modules/, .gradle/, .idea/
- ✗ **No subir:** Archivos binarios pesados (>100 MB) sin Git LFS
- † **Advertencia:** Un PAT comprometido permite acceso completo al repositorio. Tratarlo como un password
- **Solucion:** Configurar .gitignore correctamente desde el inicio y revocar tokens comprometidos en GitHub Settings

8 Referencias y Recursos

- [Documentacion oficial de Git](#)
- [GitHub Docs - Get Started](#)
- [Managing Personal Access Tokens \(PAT\)](#)
- [Coleccion oficial de plantillas .gitignore](#)
- [Conventional Commits - Estandar de mensajes de commit](#)

Doc ID: GIT-GITHUB-TERMINAL-2026-001 | Version: 1.0 | Status: Production Ready
Este documento es parte del protocolo estandar de control de versiones de Illarek-Lab